

Collegium Witelona Uczelnia Państwowa

Zaawansowane metody programowania

Aplikacja webowa Alkozon

Dokumentacja projektowa

Temat: Firma produkująca alkohol — system obsługi zamówień

Autor:
Jakub Janiec

Repozytorium: <https://github.com/Jakub137/web-Alkozon>

Wdrożenie: <https://web-alkozon.vercel.app/>

Legnica, 2026

Spis treści

1	Wstęp	1
2	Założenia projektowe	2
3	Stack technologiczny	3
3.1	Wybór technologii	3
3.2	Testy i CI	3
4	Architektura i opis działania	4
4.1	Struktura katalogów	4
4.2	Przepływ danych	4
5	Funkcjonalności klienta webowego	5
5.1	Sklep i koszyk	5
5.2	Zamówienie własne	5
5.3	Śledzenie zamówienia	5
5.4	Konto, gość, język, motyw	5
5.5	Realtime	6
6	Bezpieczeństwo	7
6.1	Zakres odpowiedzialności warstwy webowej	7
6.2	Walidacja formularzy i ograniczanie niebezpiecznego wejścia	8
6.3	Bramka wieku i kontrola dostępu w interfejsie	8
6.4	Obsługa sesji i tokenów	8
6.5	Ochrona procesu składania zamówienia	9
6.6	Obsługa błędów API w warstwie prezentacji	9
6.7	Nagłówki bezpieczeństwa HTTP (Next.js)	9
6.8	Elementy poza zakresem aplikacji webowej	10
6.9	Weryfikacja testami automatycznymi	10
7	Jakość kodu — Prettier i ESLint	11
7.1	Prettier — spójny wygląd plików	11
7.2	ESLint — reguły React/Next	11
7.3	Podsumowanie rozdziału	11
8	Wzorce projektowe	12
8.1	Wzorce strukturalne	12
8.1.1	Facade (fasada)	12
8.1.2	Adapter (adapter)	13

8.1.3	Composite (kompozyt)	13
8.2	Wzorce behawioralne	13
8.2.1	Observer (obserwator)	13
8.2.2	Strategy (strategia)	14
8.2.3	State (stan)	14
8.3	Wzorzec kreacyjny	14
8.3.1	Factory Method (metoda fabryczna)	14
9	Testowanie	15
10	Integracja z API i wdrożenie	16
10.1	Konfiguracja połączenia z API	16
10.2	Uruchomienie lokalne	16
10.3	Wdrożenie produkcyjne	17
11	Wygląd interfejsu	18
12	Podsumowanie	32

Rozdział 1

Wstęp

W ramach projektu zespołowego opracowano system **Alkozon** — platformę obsługi sprzedaży produktów alkoholowych. Użytkownik może przeglądać katalog, składać zamówienia standardowe lub skonfigurować własny trunek, śledzić status realizacji, założyć konto lub korzystać z serwisu jako gość. Sprzedaż alkoholu wymaga potwierdzenia pełnoletności.

Niniejsze sprawozdanie dotyczy **wyłącznie aplikacji webowej** (**web-Alkozon**) — klienta działającego w przeglądarce, zrealizowanego w technologii Next.js i komunikującego się z centralnym API systemu. Pozostałe komponenty rozwiązania (aplikacja mobilna, aplikacja desktopowa) są opracowywane przez innych członków zespołu i nie stanowią przedmiotu tego dokumentu.

Zakres opracowania obejmuje: założenia funkcjonalne, stos technologiczny, architekturę, bezpieczeństwo warstwy prezentacji, narzędzia jakości kodu (Prettier, ESLint), wzorce projektowe wraz z opisem ich zastosowania w kodzie, testy automatyczne, wdrożenie aplikacji webowej oraz dokumentację interfejsu użytkownika.

Rozdział 2

Założenia projektowe

Poniżej przedstawiono wymagania funkcjonalne aplikacji webowej wynikające ze specyfikacji projektu (tabela 2.1).

Tabela 2.1: Założenia funkcjonalne klienta webowego

Lp.	Założenie
1	Zamawianie produktów (sklep, koszyk, checkout).
2	Śledzenie zamówienia po numerze i e-mailu.
3	Własne zamówienie — konfigurator trunku.
4	Rejestracja i logowanie (CUSTOMER).
5	Gość + weryfikacja wieku (bez 18+ brak zakupu).
6	Informacje o alkoholach (FAQ, historia).
7	Tryb ciemny, język PL / EN.
8	Responsywny interfejs (RWD).
9	Połączenie z API; status zamówienia na żywo (STOMP).

Część mechanizmów bezpieczeństwa (np. hashowanie haseł, autoryzacja po stronie serwera) jest realizowana w API. Aplikacja webowa uzupełnia je walidacją danych wejściowych w formularzach oraz kontrolą dostępu w interfejsie użytkownika.

Rozdział 3

Stack technologiczny

3.1 Wybór technologii

Warstwa prezentacji została zrealizowana w oparciu o Next.js, TypeScript, React oraz Tailwind CSS (tabela 3.1).

Tabela 3.1: Stos technologiczny web-Alkozon

Warstwa	Technologia
Framework	Next.js 16 (App Router) — routing, szybkie ładowanie stron.
UI	React 19, TypeScript 5 — komponenty i typowanie.
Style	Tailwind CSS v4 — responsywność bez pisania dużych plików CSS.
Formularze	react-hook-form + Zod — walidacja zgodna z API.
Realtime	@stomp/stompjs — WebSocket, zmiany statusu zamówienia.
Ikony	lucide-react.

3.2 Testy i CI

Testy jednostkowe i integracyjne wykonuje się przy użyciu Vitest oraz Testing Library; testy end-to-end — Playwright. Repozytorium zawiera workflow GitHub Actions (lint, build, testy w kilku jobach).

Rozdział 4

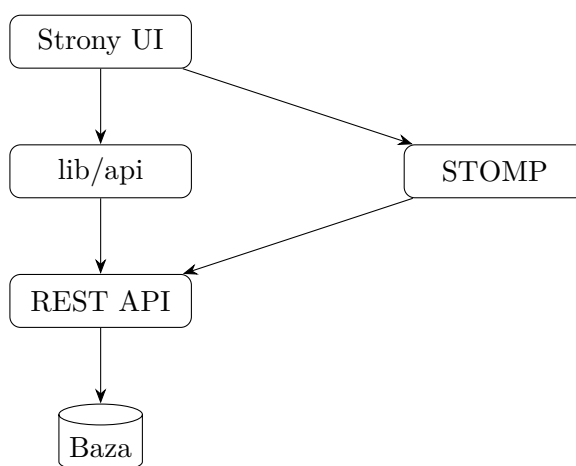
Architektura i opis działania

4.1 Struktura katalogów

Struktura kodu została podzielona tak, aby komponenty stron w `src/app/` ograniczały się do prezentacji, a logika komunikacji z API znajdowała się w `src/lib/api/`:

```
web-Alkozon/  
  src/app/      # strony (shop, cart, login, ...)  
  src/components/ # przyciski, karty, naglowek  
  src/context/   # Auth, Cart, Language, Theme, Age  
  src/lib/api/    # fasada HTTP  
  src/lib/realtime/ # STOMP  
  tests/         # unit, integracja, e2e
```

4.2 Przepływ danych



Rysunek 4.1: Uproszczony przepływ danych aplikacji webowej

Użytkownik wchodzi na stronę, potwierdza wiek, potem może przejść do sklepu. Żeby złożyć zamówienie, musi być zalogowany jako **CUSTOMER**. Śledzenie zamówienia działa też bez konta (numer + e-mail).

Rozdział 5

Funkcjonalności klienta webowego

5.1 Sklep i koszyk

Katalog dostępny jest pod adresem `/shop`; stan koszyka przechowywany jest w `CartContext`. Złożenie zamówienia wymaga konta z rolą `CUSTOMER` oraz potwierdzonej pełnoletności.

5.2 Zamówienie własne

Strona `/custom-order` umożliwia konfigurację trunku. Przy wielu pozycjach w koszyku generowany jest **osobny numer zamówienia klienckiego** na każdą sztukę (implementacja w `cart/page.tsx`), co rozwiązuje konflikt HTTP 409 przy ponownym użyciu tego samego numeru.

5.3 Śledzenie zamówienia

Strona `/order-status` realizuje śledzenie publiczne: najpierw zapytanie `/api/orders/track`, a w przypadku braku wyniku — `/api/custom-orders/track`.

5.4 Konto, gość, język, motyw

Rejestracja obejmuje walidację hasła (Zod). Logowanie oparte jest na tokenach JWT; dodatkowo zastosowano licznik nieudanych prób w przeglądarce (maks. 5). Gość przechodzi przez modal weryfikacji wieku — bez potwierdzenia pełnoletności zakup alkoholu jest zablokowany. Dostępne są sekcje FAQ i historia oraz przełącznik języka PL/EN i trybu ciemnego.

5.5 Realtime

Po zalogowaniu aplikacja może subskrybować zdarzenia STOMP (`subscribeOrderStatusUpdates`) — widok listy zamówień i statusu aktualizuje się bez przeładowania strony.

Rozdział 6

Bezpieczeństwo

Bezpieczeństwo systemu Alkozon opiera się na współpracy warstwy webowej z centralnym API. Niniejszy rozdział opisuje mechanizmy realizowane w aplikacji Next.js udostępnianej użytkownikowi w przeglądarce: jak chronią dane wprowadzane w formularzach, jak zarządzają sesją oraz jakie zabezpieczenia transportowe stosuje hosting. Hashowanie haseł, podpisywanie tokenów JWT, kontrola ról po stronie serwera i operacje na bazie danych należą do API — aplikacja webowa je wspiera poprzez walidację wejścia, poprawną obsługę odpowiedzi oraz ograniczenie dostępu w interfejsie.

6.1 Zakres odpowiedzialności warstwy webowej

Warstwa prezentacji nie zastępuje zabezpieczeń serwera — pełni rolę pierwszej linii obrony i warstwy użyteczności. Użytkownik otrzymuje natychmiastową informację o błędzie (np. zbyt słabe hasło) zanim żądanie trafi do API; w koszyku i przy weryfikacji wieku interfejs uniemożliwia przejście do zakupu alkoholu osobom nieuprawnionym. Krytyczne decyzje autoryzacyjne muszą być powtórzone po stronie API; web zmniejsza liczbę błędnych żądań i utrudnia obejście reguł biznesowych wyłącznie manipulacją DOM w przeglądarce.

Tabela 6.1: Bezpieczeństwo — skrót realizacji w aplikacji webowej

Obszar	Realizacja w aplikacji webowej
Walidacja danych	Zod, <code>react-hook-form</code> , wzorce zgodne z API (<code>backendPatterns.ts</code>).
Kontrola dostępu (UI)	Rola <code>CUSTOMER</code> , potwierdzona pełnoletność, bramka wieku 18+.
Sesja	<code>AuthContext</code> , tokeny Bearer, odświeżanie przy 401.
Ochrona sesji	Auto-wylogowanie po 15 min bezczynności; synchronizacja wylogowania między kartami.
Nagłówki HTTP	HSTS, <code>X-Frame-Options</code> , CSP w trybie <code>Report-Only</code> (<code>next.config.ts</code>).
Transport	HTTPS na Vercel; STOMP przez <code>wss</code> .

6.2 Walidacja formularzy i ograniczanie niebezpiecznego wejścia

Formularze logowania i rejestracji korzystają z `react-hook-form` oraz schematów `Zod`. Przed wysłaniem żądania do API sprawdzane są m.in.: poprawny format adresu e-mail, obecność hasła przy logowaniu, wymagania dotyczące siły hasła przy rejestracji, dozwolony zestaw znaków w nazwie użytkownika oraz potwierdzenie pełnoletności. Wzorce walidacyjne (np. `SAFE_TEXT_REGEX`) są zdefiniowane w `src/lib/validation/backendPatterns.ts` i są zgodne z regułami backendu, dzięki czemu komunikaty błędów są spójne w całym systemie.

W polach tekstowych blokowane są znaki niedozwolone tam, gdzie API stosuje analogiczne ograniczenia. Nie jest to jedyna ochrona przed atakami injection — aplikacja webowa nie wykonuje zapytań SQL — lecz ogranicza przypadkowe lub złośliwe wpisy już na etapie wprowadzania danych i poprawia doświadczenie użytkownika. Ostateczna weryfikacja DTO odbywa się po stronie serwera API.

6.3 Bramka wieku i kontrola dostępu w interfejsie

Ze względu na charakter produktu (alkohol) zastosowano **bramkę wieku** (`AgeContext`). Użytkownik musi potwierdzić pełnoletność; oznaczenie jako niepełnoletni skutkuje wyświetleniem strony z ograniczeniem zamiast ścieżki zakupowej. Gość przechodzi przez modal weryfikacji przed korzystaniem ze sklepu.

W koszyku dodatkowo weryfikowane są rola użytkownika (`CUSTOMER`) oraz status pełnoletności — dopiero wtedy odblokowany zostaje formularz danych dostawy i możliwość złożenia zamówienia. Mechanizm ten realizuje reguły RBAC na poziomie prezentacji: nawet przy poprawnym tokenie użytkownik bez odpowiedniej roli lub bez potwierdzonego wieku nie przejdzie przez interfejs do finalizacji transakcji (serwer API musi te same warunki egzekwować przy przyjmowaniu zamówienia).

6.4 Obsługa sesji i tokenów

Stan zalogowania utrzymywany jest w `AuthContext`. Żądania wymagające autoryzacji przechodzą przez warstwę `authorizedRequest`, która dołącza access token w nagłówku `Authorization`, a w razie odpowiedzi 401 próbuje odświeżyć sesję przy użyciu refresh tokena i ponowić zapytanie z nowym access tokenem. Generowanie, podpisywanie i weryfikacja tokenów JWT należą do backendu; aplikacja webowa przechowuje je po stronie klienta i przekazuje zgodnie z kontraktem API.

Dodatkowo zaimplementowano **auto-wylogowanie** po 15 minutach braku aktywności (ruch myszy, kliknięcia, klawiatura, przewijanie). Ogranicza to ryzyko pozostawienia otwartej sesji na współdzielonym komputerze.

Na etapie finalnym projektu wprowadzono **synchronizację wylogowania między kartami** przeglądarki (zdarzenie `storage`): wylogowanie w jednej karcie czyści stan sesji w pozostałych i informuje użytkownika komunikatem. Zmiana nie modyfikuje kontraktu z API — dotyczy wyłącznie spójności stanu po stronie klienta.

Przy logowaniu stosowany jest pomocniczy **licznik nieudanych prób** w przeglądarce (maksymalnie 5) — uzupełnia on ewentualny rate limiting po stronie serwera i ogranicza wielokrotne wysyłanie błędnych danych z tego samego urządzenia.

6.5 Ochrona procesu składania zamówienia

W module koszyka zastosowano blokadę `submitLockRef`, która uniemożliwia wielokrotne, bardzo szybkie kliknięcie przycisku złożenia zamówienia i tym samym podwójne wysłanie tego samego żądania. Przy konflikcie numeru zamówienia (odpowiedź 409) frontend generuje nowy numer klienta i ponawia próbę — użytkownik nie musi ręcznie poprawiać danych po kolizji identyfikatora.

6.6 Obsługa błędów API w warstwie prezentacji

Klient HTTP mapuje istotne kody statusu na komunikaty zrozumiałe w UI (przez adapter błędów i słowniki PL/EN), zamiast prezentować surową treść techniczną. W praktyce obsługiwane są m.in.: 401 (brak lub wygasła autoryzacja), 403 (brak uprawnień), 409 (konflikt numeru zamówienia), 429 (zbyt wiele prób) oraz 500+ (błąd serwera). Dzięki temu użytkownik wie, czy powinien się ponownie zalogować, odczekać, czy skontaktować się z obsługą — bez analizowania odpowiedzi JSON.

6.7 Nagłówki bezpieczeństwa HTTP (Next.js)

W pliku `next.config.ts` skonfigurowano nagłówki bezpieczeństwa dla wszystkich ścieżek aplikacji:

- **Strict-Transport-Security** — wymusza korzystanie z HTTPS w przeglądarce po pierwszym połączeniu;
- **X-Content-Type-Options: nosniff** — ogranicza MIME sniffing;
- **X-Frame-Options: DENY** — chroni przed osadzaniem strony w obcych ramkach (clickjacking);
- **Referrer-Policy: strict-origin-when-cross-origin** — ogranicza wyciek pełnych adresów URL w nagłówku referer;
- **Permissions-Policy** — wyłącza nieużywane uprawnienia przeglądarki (kamera, mikrofon, geolokalizacja, płatności).

Dodatkowo ustawiono **Content-Security-Policy-Report-Only** — polityka CSP w trybie *tylko raportowania*, bez blokowania zasobów. Na końcowym etapie projektu unikano twardego CSP, które mogłoby zakłócić połączenie z API, WebSocketami STOMP, Firebase lub zasobami Next.js na Vercel; tryb **Report-Only** dokumentuje przygotowanie polityki bez ryzyka awarii wdrożonej aplikacji.

6.8 Elementy poza zakresem aplikacji webowej

Poniższe mechanizmy są istotne dla całego systemu, lecz nie są implementowane w repozytorium **web-Alkozon**: hashowanie haseł, weryfikacja JWT po stronie serwera, RBAC w API, rate limiting serwera, walidacja DTO na backendzie, przechowywanie danych w bazie, konfiguracja CORS API, sekrety serwera oraz firewall/WAF infrastruktury API. Aplikacja webowa integruje się z nimi przez poprawne żądania HTTP i obsługę odpowiedzi.

Na platformie **Vercel** zapewniane są m.in. certyfikat TLS, obsługa HTTPS oraz konfiguracja zmiennych środowiskowych (`NEXT_PUBLIC_*`). W ramach oddawania projektu nie wprowadzano ryzykownych zmian integracji (migracja na cookies zamiast tokenów, twarde CSP blokujące zasoby, modyfikacja payloadów checkoutu ani adresów API) — wzmocniono warstwę prezentacji i nagłówki bez zakłócania działającego połączenia z backendem.

6.9 Weryfikacja testami automatycznymi

Zachowania związane z bezpieczeństwem są objęte testami w pakiecie `npm run test:run` oraz scenariuszami E2E (Playwright): m.in. walidacja haseł, `AuthContext`, modal weryfikacji wieku, koszyk, auto-logout oraz obsługa błędów klienta HTTP przy śledzeniu zamówień. Testy ułatwiają wykrycie regresji przy kolejnych zmianach w kodzie.

Rozdział 7

Jakość kodu — Prettier i ESLint

Zgodnie z wymaganiami przedmiotu zwrócono uwagę na *czysty kod*. Oprócz podziału modułów zastosowano narzędzia wspierające spójność stylu i wykrywanie błędów.

7.1 Prettier — spójny wygląd plików

Prettier służy do automatycznego formatowania kodu źródłowego (wcięcia, cudzysłowy, łamanie linii w JSX). Nie weryfikuje logiki programu, lecz zapewnia jednolitą strukturę plików w repozytorium. Uruchomienie: `npm run format` lub weryfikacja: `npm run format:check`.

7.2 ESLint — reguły React/Next

ESLint analizuje poprawność idiomów React i Next.js (m.in. reguły hooków). Uruchomienie: `npm run lint`. Pakiet **eslint-config-prettier** wyłącza reguły formatowania kolidujące z Prettierem — formatter odpowiada za układ, linter za semantykę kodu.

7.3 Podsumowanie rozdziału

Prettier odpowiada za jednolity format, ESLint — za zgodność z konwencjami frameworka. Uzupełniają one testy automatyczne, podnosząc czytelność i jakość utrzymania kodu.

Rozdział 8

Wzorce projektowe

Poniżej przedstawiono wzorce projektowe z katalogu GoF (kreacyjne, strukturalne, behawioralne), które występują w kodzie aplikacji webowej. Dla każdego wzorca podano definicję, zastosowanie w projekcie oraz uzasadnienie wyboru.

8.1 Wzorce strukturalne

8.1.1 Facade (fasada)

Od czego jest: Fasada to “jedno wejście” do bardziej skomplikowanego podsystemu. Zamiast wołać wiele niskopoziomowych operacji, klient korzysta z jednej uproszczonej warstwy.

Zastosowanie w projekcie: Cały dostęp do REST API realizowany jest przez folder `src/lib/api/`. Funkcja `apiRequest()` w `client.ts` realizuje wywołania `fetch`, składanie adresu URL na podstawie `NEXT_PUBLIC_API_URL`, ustawienie nagłówka `Authorization` oraz parsowanie błędów JSON. Osobne pliki (`orders.ts`, `auth.ts`, `customOrders.ts`) eksportują gotowe funkcje typu “pobierz zamówienia”, “zaloguj”.

Po co: Komponenty stron (koszyk, sklep, logowanie) nie powielają tej samej obsługi HTTP. Jak zmieni się sposób zwracania błędu z API, poprawiam głównie `lib/api`, a nie dziesiątki komponentów.

8.1.2 Adapter (adapter)

Od czego jest: Adapter dopasowuje jeden interfejs do drugiego — gdy “źródło” mówi innym językiem niż “odbiorca”.

Zastosowanie w projekcie: Serwer API zwraca komunikaty po angielsku lub w stałych kodach (`Invalid credentials`). UI ma pokazywać teksty po polsku lub angielsku z moich słowników. Plik `mapApiErrorMessage.ts` mapuje odpowiedź API na klucze ze słownika (`dict.auth.errors`) — użytkownik widzi zrozumiały komunikat, a nie surowy JSON.

Po co: Oddzielam protokół API od warstwy prezentacji.

8.1.3 Composite (kompozyt)

Od czego jest: Kompozyt traktuje pojedyncze obiekty i ich grupy tak samo — budujemy drzewo z mniejszych części.

Zastosowanie w projekcie: W React całe UI to drzewo komponentów. W `layout.tsx` jest szkielet strony (nagłówek, menu), a `{children}` to treść konkretnej podstrony. W `providers.tsx` owijam aplikację kilkoma kontekstami (`ThemeProvider`, `AuthProvider`, `CartProvider` itd.) — dla stron wygląda to jak jeden “korzeń”, choć w środku jest wiele providerów.

Po co: Nie muszę powtarzać nagłówka na każdej stronie; Next.js sam składa `layout + page`.

8.2 Wzorce behawioralne

8.2.1 Observer (obserwator)

Od czego jest: Jeden obiekt publikuje zdarzenia, a wielu obserwatorów może się **subskrybować** i reagować bez ciągłego odpytywania “czy coś się zmieniło”.

Zastosowanie (STOMP): W `lib/realtime/orderUpdates.ts` funkcja `subscribeOrderStatusUp` łączy się z WebSoketem, subskrybuje kolejkę `/user/queue/order-updates` i przy każdej wiadomości woła przekazany callback `onEvent`. Strony `order-status` i `my-orders` podpinają się pod to i odświeżają widok statusu.

Zastosowanie (powiadomienia w interfejsie): `NotificationContext` trzyma listę powiadomień; komponenty mogą dodać wpis, a `NotificationContainer` je wyświetla — klasyczny `publish/subscribe` w obrębie Reacta.

Po co: Status zamówienia może zmienić się na magazynie lub u kuriera — użytkownik widzi update bez F5.

8.2.2 Strategy (strategia)

Od czego jest: Strategia pozwala **wymieniać algorytm** (tu: sposób dostarczenia treści) bez przerabiania całej klasy/komponentu.

Zastosowanie w projekcie: `LanguageContext` trzyma `lang` (`pl` lub `en`) i wystawia `dict` — albo `dictionaries.pl`, albo `dictionaries.en`. Komponenty biorą teksty z `dict`, a przełącznik języka tylko zmienia strategię słownika.

Po co: Nie kopiuję całych stron na PL i EN; zmieniam jeden kontekst.

8.2.3 State (stan)

Od czego jest: Obiekt zmienia zachowanie, gdy zmienia się jego stan wewnętrzny (maszyna stanów).

Zastosowanie w projekcie: `AgeContext` przechowuje np. `ageStatus`: niezweryfikowany, pełnoletni, niepełnoletni. Od tego zależy, czy pokażę modal, czy `UnderageRestrictedPage`, czy pozwolę wejść do koszyku z alkoholem.

Po co: Reguły prawne (18+) są w jednym miejscu, a nie rozrzucone po dziesiątkach warunków `if` w każdym pliku.

8.3 Wzorzec kreacyjny

8.3.1 Factory Method (metoda fabryczna)

Od czego jest: Podklasa lub funkcja decyduje, **jaki obiekt** (tu: jaka wartość) ma powstać, żeby ukryć szczegóły tworzenia.

Zastosowanie w projekcie: W koszyku `generateOrderNumber()` w `cart/page.tsx` generuje unikalny numer zamówienia klienckiego w jednym miejscu. Przy wielu sztukach `custom order` wołam ją wielokrotnie; przy konflikcie 409 generuję nowy numer w pętli.

Po co: Nie kopiuję losowania numeru w trzech miejscach — jak zmieni się format numeru, poprawiam jedną funkcję.

Rozdział 9

Testowanie

Testy uruchamiane są lokalnie oraz w procesie CI (GitHub Actions). Podział przedstawia tabela 9.1.

Tabela 9.1: Testy automatyczne web-Alkozon

Typ	Gdzie	Komenda	Liczba
Unit	tests/unit/	npm run test:unit	408
Integracja UI	tests/integration/	test:integration-ui	6
Integracja API	tests/integration/api/	test:integration-api	12
E2E	Playwright	npm run test:e2e	osobne scenariusze
Razem (bez E2E)	—	npm run test:run	426

Zakres obejmuje m.in. walidację haseł, `AuthContext`, koszyk, modal weryfikacji wieku oraz logikę klienta HTTP (śledzenie zamówień sklepowych i custom). Testy ułatwiają wykrywanie regresji przy zmianach w kodzie.

Rozdział 10

Integracja z API i wdrożenie

10.1 Konfiguracja połączenia z API

Aplikacja webowa komunikuje się z zewnętrznym serwerem API za pomocą zmiennych środowiskowych (plik `.env.local` lokalnie, panel Vercel na produkcji).

NEXT_PUBLIC_API_URL — bazowy adres URL serwera API (backendu), bez ukośnika na końcu, np. `https://api-alcozon.onrender.com` lub lokalnie `http://localhost:8080`. W kodzie (`src/lib/api/client.ts`) żądania HTTP budowane są jako suma tej wartości i ścieżki endpointu (np. `/api/orders`). **To nie jest adres samej aplikacji webowej** — adres strony dla użytkownika to osobna domena (patrz wdrożenie).

NEXT_PUBLIC_WS_URL (opcjonalnie) — pełny adres WebSocket do powiadomień STOMP; jeśli brak, aplikacja próbuje wywnioskować go z **NEXT_PUBLIC_API_URL**.

10.2 Uruchomienie lokalne

```
cd web-Alcozon
npm install
cp .env.example .env.local
npm run dev
```

W `.env.local` należy ustawić **NEXT_PUBLIC_API_URL** na działający serwer API. Aplikacja webowa uruchamia się pod `http://localhost:3000`.

10.3 Wdrożenie produkcyjne

Aplikacja webowa została wdrożona na platformie **Vercel**. Publiczny adres:

<https://web-alkozon.vercel.app/>

W panelu Vercel skonfigurowano zmienne środowiskowe (w tym `NEXT_PUBLIC_API_URL` wskazujące na działające API systemu). Wdrożenie realizowane jest z repozytorium Git — po pushu uruchamiany jest build Next.js oraz publikacja wersji produkcyjnej. Komunikacja z przeglądarką odbywa się przez HTTPS.

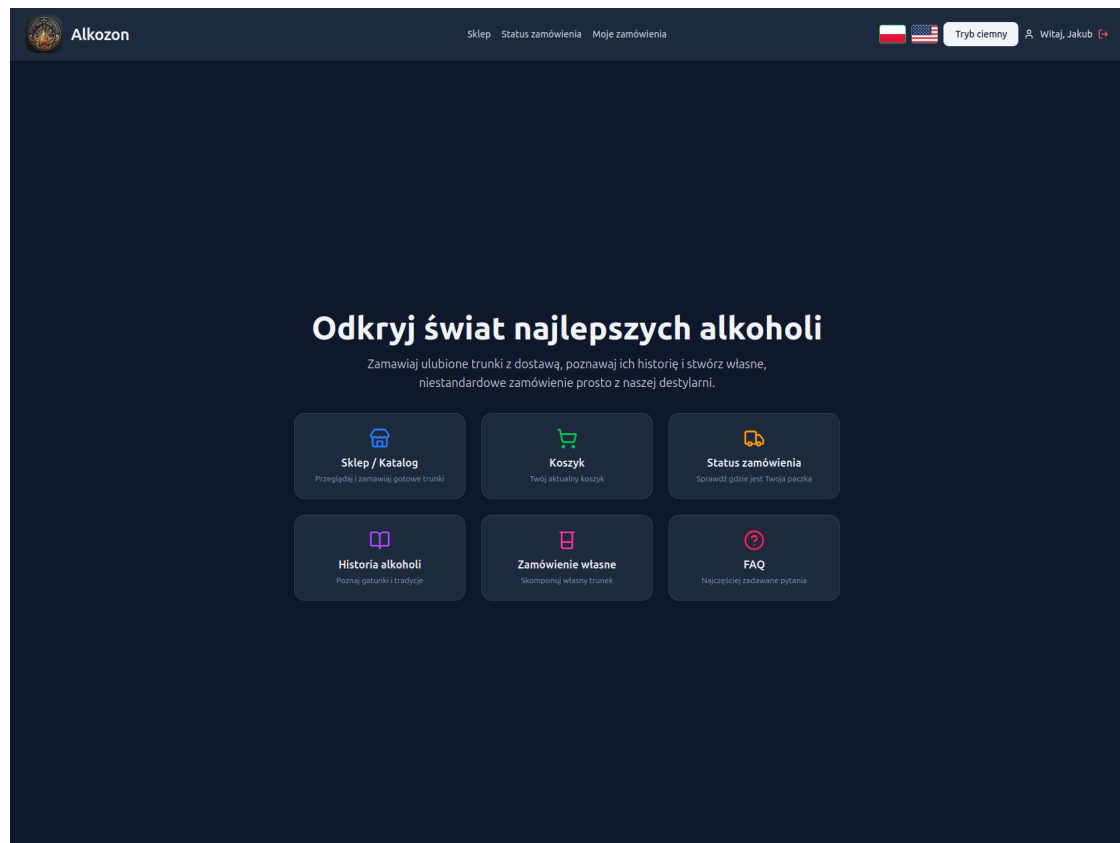
Przed oddaniem projektu zweryfikowano ścieżki: rejestracja, złożenie zamówienia oraz śledzenie statusu na środowisku produkcyjnym.

Rozdział 11

Wygląd interfejsu

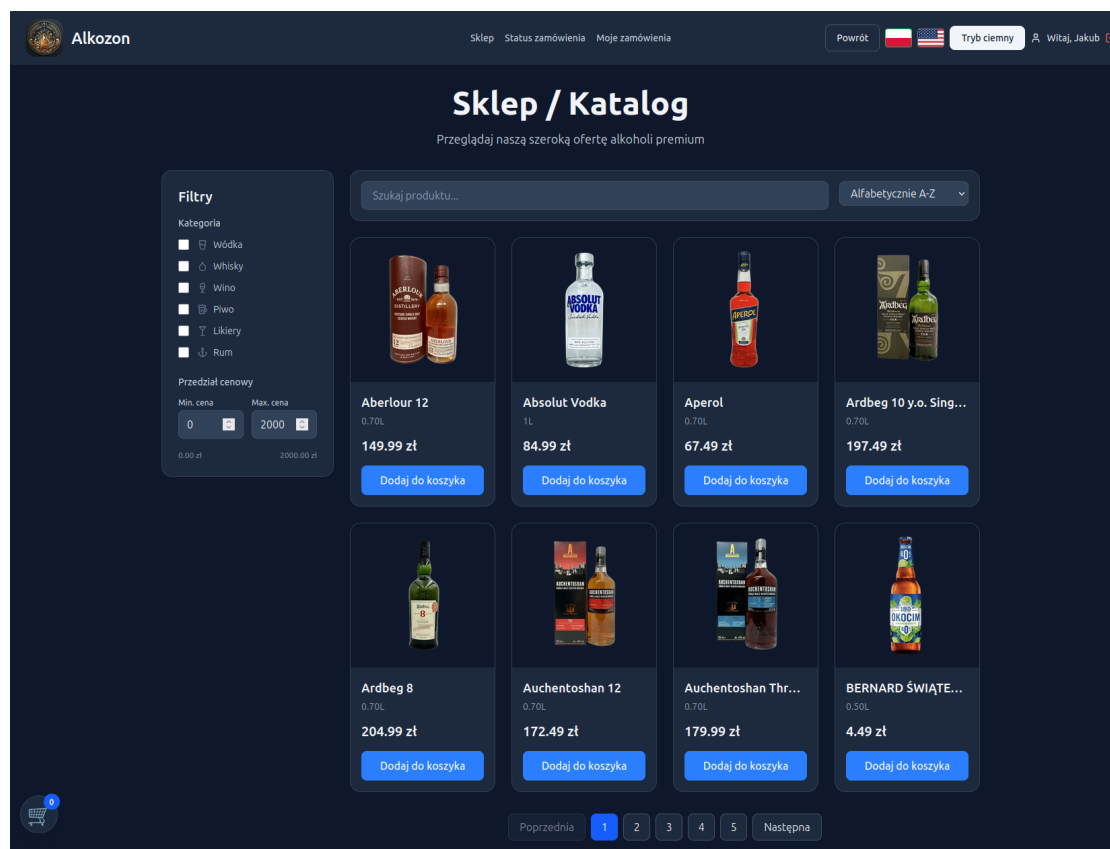
Poniżej przedstawiono zrzuty ekranu wdrożonej aplikacji webowej (<https://web-alkozon.vercel.app/>). Dla wybranych funkcji pokazano kilka etapów (np. konfiguracja zamówienia własnego). Każda podsekcja zaczyna się na nowej stronie, aby zachować czytelny układ w PDF.

Strona główna



Rysunek 11.1: Strona główna — nawigacja i wejście do modułów aplikacji

Sklep



Rysunek 11.2: Katalog produktów

Koszyk

Podsumowanie zakupów

Liczba elementów: 4

Aperol 0.70L	x1	67.49 zł	Usuń
BERNARD ŚWIĄTECZNY LAGER 0.50L	x1	4.49 zł	Usuń
Captain Morgan Spiced Gold 0.70L	x1	79.99 zł	Usuń
Carolans Irish Cream 0.70L	x1	82.49 zł	Usuń
Suma		234.46 zł	

Dane do dostawy

Metoda płatności: Płatność przy odbiorze

Imię

Nazwisko

Jakub

Kowalski

Adres

ul. Testowa 12/4

Miasto

Kod pocztowy

Warszawa

00-000

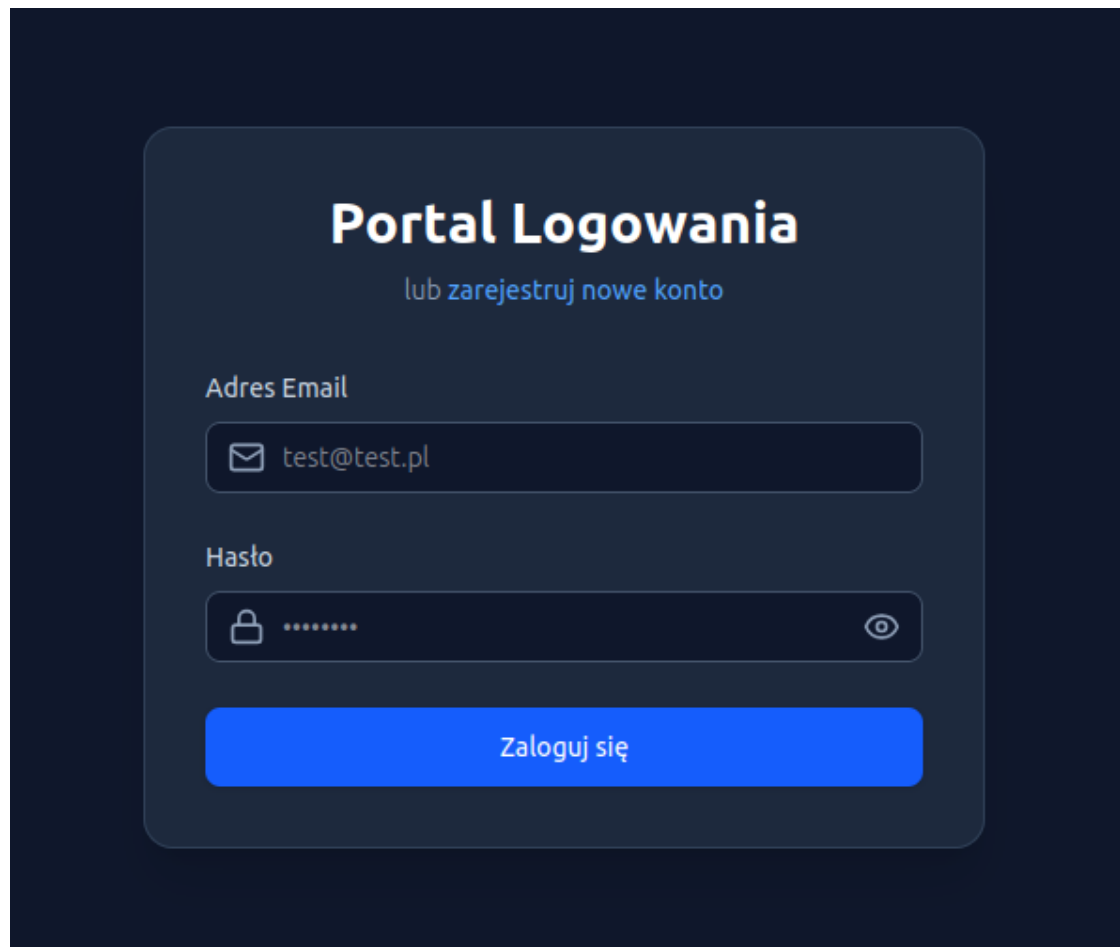
Uwagi dla dostawcy

Np. proszę zadzwonić domofonem

Złóż zamówienie

Rysunek 11.3: Koszyk oraz formularz danych zamówienia

Logowanie i rejestracja



Portal Logowania
lub zarejestruj nowe konto

Adres Email

Hasło



[Zaloguj się](#)

Rysunek 11.4: Formularz logowania użytkownika

Weryfikacja & Rejestracja

lub zaloguj się

Nazwa Użytkownika (Login)

 Janek123

Adres Email

 jan@kowalski.pl

Bezpieczne Hasło

Walidacja jak w API (bezpieczny tekst w nazwie, hasło z małą i wielką literą, cyfrą oraz znakiem @\$!%*?&).

☐ Potwierdzam, że mam ukończone 18 lat i mogę założyć konto klienta.

Zarejestruj Konto

Rysunek 11.5: Formularz rejestracji z walidacją hasła

Zamówienie własne (konfigurator)

Zamówienie własne

Skonfiguruj swój autorski trunek i sprawdź orientacyjną cenę w czasie rzeczywistym.

1. Baza

2. Smak

3. Wykończenie

Krok 1: Baza trunku

Wybierz bazę

Wódka

Whisky

Wino

Likiery

Wybierz pojemność

0.5L

0.7L

1.0L

Moc alkoholu: 40%

35%

65%

Wstecz

Dalej

Podsumowanie konfiguracji

Baza	Whisky
Pojemność	0.7L
Moc	40%
Intensywność	3
Liczba smaków	0

Cena bazowa	95.00 zł
Po pojemności	95.00 zł
Korekta mocy	+0.00 zł
Dopłata za profile	+0.00 zł
Mnożnik intensywności	x1.07

Cena **101.99 zł**

Rysunek 11.6: Zamówienie własne — etap 1 konfiguracji trunku

Zamówienie własne

Skonfiguruj swój autorski trunek i sprawdź orientacyjną cenę w czasie rzeczywistym.

1. Baza

2. Smak

3. Wykończenie

Krok 2: Charakter smaku

Profile smakowe

Słodki

Wytrawny

Owocowy

Dymny

Ziołowy

Beczkowy

Intensywność: 3

1

5

Wstecz

Dalej

Podsumowanie konfiguracji

Baza	Whisky
Pojemność	0.7L
Moc	40%
Intensywność	3
Liczba smaków	0

Cena bazowa	95.00 zł
Po pojemności	95.00 zł
Korekta mocy	+0.00 zł
Dopłata za profile	+0.00 zł
Mnożnik intensywności	x1.07

Cena **101.99 zł**

Rysunek 11.7: Zamówienie własne — etap 2

Zamówienie własne

Skonfiguruj swój autorski trunek i sprawdź orientacyjną cenę w czasie rzeczywistym.

1. Baza

2. Smak

3. Wykończenie

Krok 3: Wykończenie i podsumowanie

Nazwa własna trunku

Nazwa powinna mieć co najmniej 3 znaki. 0/20

Notatka dla destylarni

0/100 słów

Wstecz

Dodaj do koszyka

Podsumowanie konfiguracji

Baza	Whisky
Pojemność	0.7L
Moc	40%
Intensywność	3
Liczba smaków	0

Cena bazowa	95.00 zł
Po pojemności	95.00 zł
Korekta mocy	+0.00 zł
Dopłata za profile	+0.00 zł
Mnożnik intensywności	x1.07

Cena **101.99 zł**

Rysunek 11.8: Zamówienie własne — etap 3 (podsumowanie / złożenie)

Śledzenie statusu zamówienia

Status zamówienia

Wpisz numer zamówienia i adres e-mail, aby sprawdzić aktualny etap realizacji.

Numer zamówienia

575174

E-mail

jakub.janiec@studenci.collegiumwitelona.pl

Sprawdź status

[Przejdź do listy: Moje zamówienia](#)

Szczegóły zamówienia

Złożone

Numer zamówienia

575174

Data złożenia

18 maj 2026

Status

Złożone

Szacowana dostawa

21 maj 2026

Postęp zamówienia

Przyjęte

W realizacji

Wysłane

Dostarczone

Co dalej?

Zamówienie jest złożone i czeka na rozpoczęcie realizacji.

Przejdź do sklepu

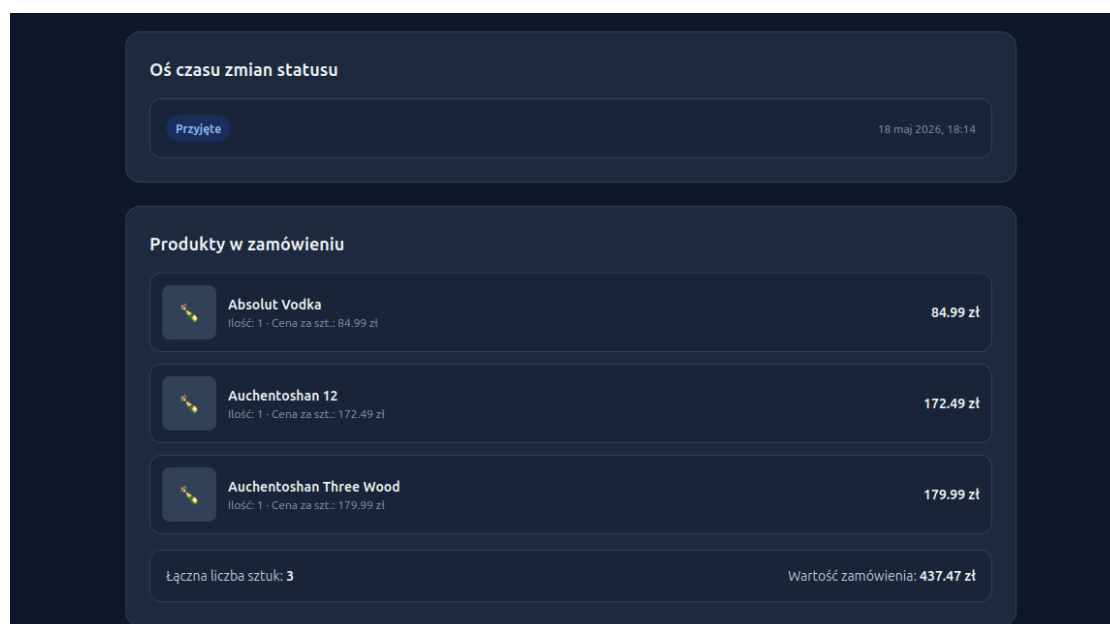
Kontakt

Oś czasu zmian statusu

Przyjęte

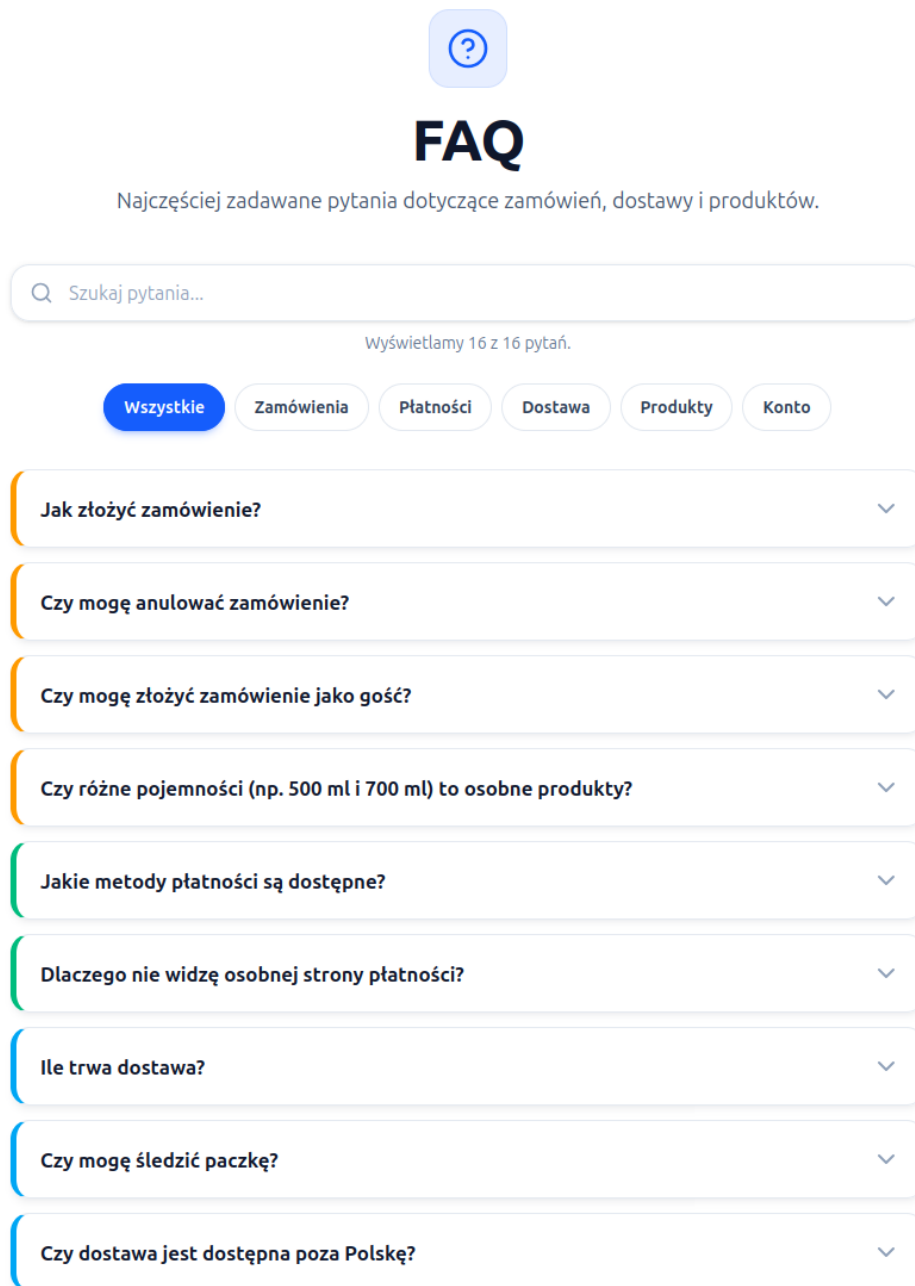
18 maj 2026, 18:14

Rysunek 11.9: Śledzenie zamówienia — wprowadzenie numeru i adresu e-mail

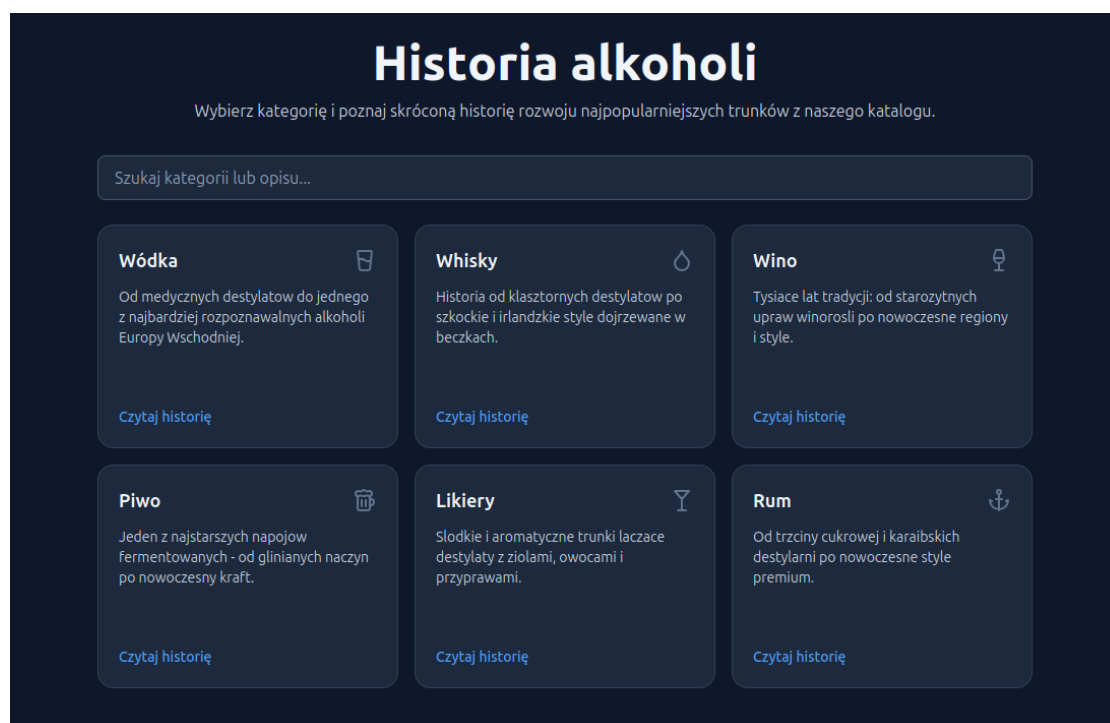


Rysunek 11.10: Śledzenie zamówienia — widok statusu realizacji oraz lista produktów

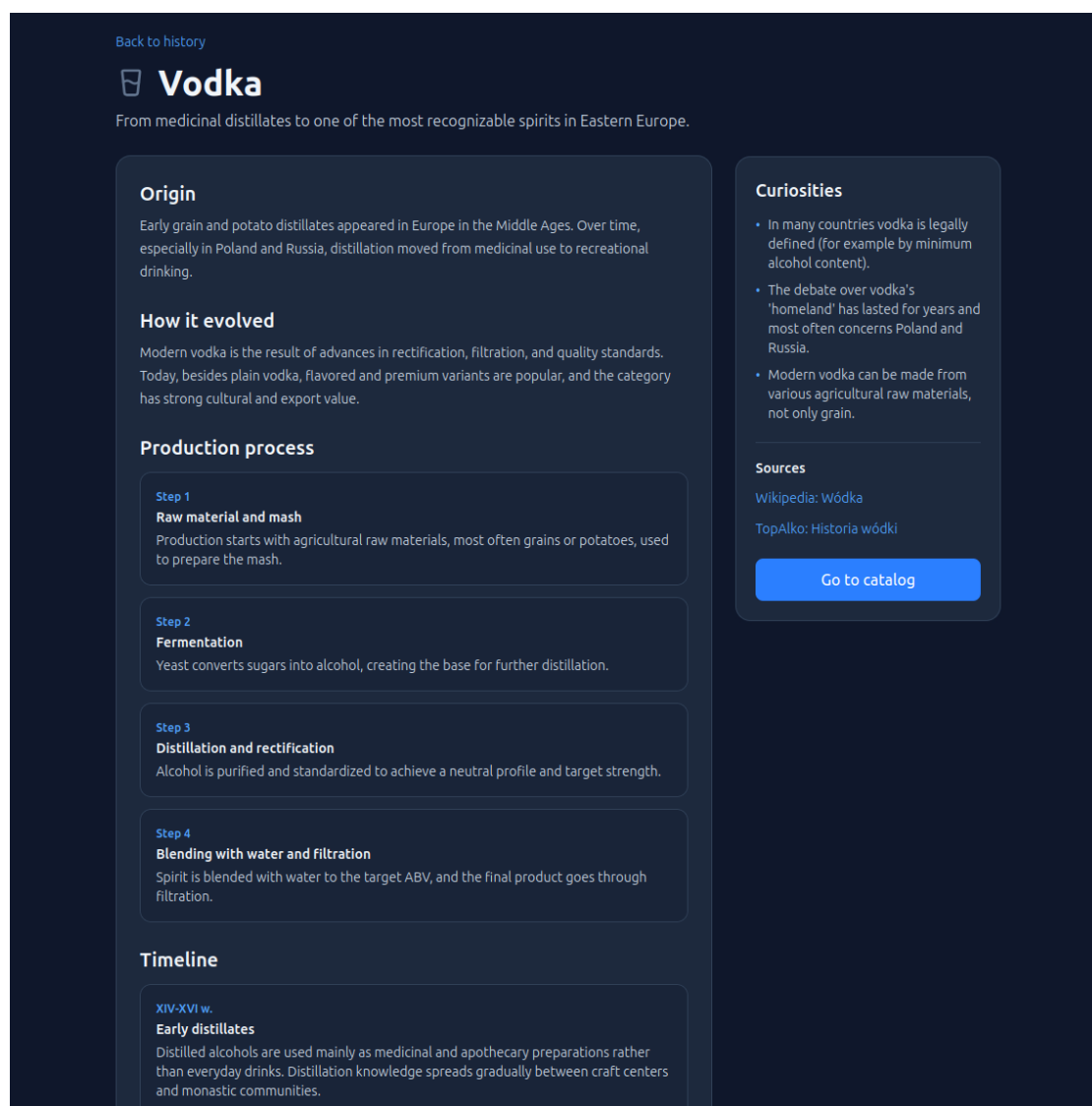
Sekcja informacyjna



Rysunek 11.11: Sekcja FAQ



Rysunek 11.12: Historia alkoholi — widok listy kategorii



Rysunek 11.13: Historia alkoholi — artykuł (fragment 1)

Timeline

XIV-XVI w.
Early distillates

Distilled alcohols are used mainly as medicinal and apothecary preparations rather than everyday drinks. Distillation knowledge spreads gradually between craft centers and monastic communities.

XVII-XVIII w.
Production growth

Distillery development increases production scale and makes the spirit more available in local trade. Vodka starts to play not only a practical role, but also a social and cultural one.

XIX-XX w.
Standardization

Advances in rectification, quality control, and production standards make spirit profiles more consistent. Distilleries move from local craft toward a more industrial production model.

XXI w.
Global market

Vodka becomes a global category with strong premium and flavored segments. Design, raw-material origin, and brand storytelling gain importance alongside alcohol strength.

Gallery



Methods of bottling vodka throughout history.



An archival view of an older man making vodka in his garden.



Historical, craft vodka production in Russia in the 20th century.



Contemporary spirits-drinking culture.

Curiosities

- In many countries vodka is legally defined (for example by minimum alcohol content).
- The debate over vodka's 'homeland' has lasted for years and most often concerns Poland and Russia.
- Modern vodka can be made from various agricultural raw materials, not only grain.

Sources

[Wikipedia: Wódka](#)

[TopAlko: Historia wódki](#)

[Go to catalog](#)

Rysunek 11.14: Historia alkoholi — artykuł (fragment 2)

Rozdział 12

Podsumowanie

W ramach projektu opracowano i wdrożono **aplikację webową Alkozon** w technologii Next.js, spełniającą założenia specyfikacji: sklep i koszyk, zamówienia własne, śledzenie zamówień, obsługę konta użytkownika, weryfikację pełnoletności, wersję językową PL/EN, tryb ciemny oraz integrację z centralnym API. Struktura kodu opiera się na warstwie `lib/api` i kontekstach React; jakość utrzymania wspierają Prettier, ESLint oraz testy automatyczne.

Zastosowano m.in. wzorce **Facade** i **Adapter** w komunikacji z API, **Observer** przy aktualizacji statusu (STOMP), **Strategy** przy internacjonalizacji oraz **State** przy weryfikacji wieku. W repozytorium zdefiniowano **426** testów w pakiecie `npm run test:run` (bez scenariuszy E2E).

Aplikacja dostępna jest pod adresem <https://web-alkozon.vercel.app/>. Po stronie warstwy webowej zastosowano m.in. walidację formularzy, bramkę wieku, obsługę sesji z odświeżaniem tokenów, auto-wylogowanie, synchronizację sesji między kartami oraz nagłówki bezpieczeństwa HTTP. Kierunki dalszego rozwoju obejmują rozszerzenie powiadomień Web Push oraz dodatkowe scenariusze testów E2E w Playwright.